mapping

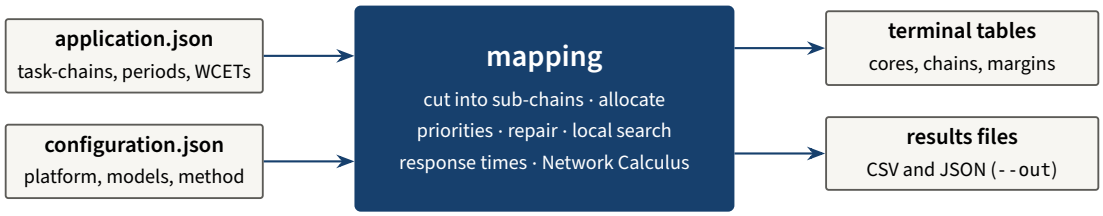
User manual

Sizing the processor cores of an automotive E/E architecture for its task-chains, with logical execution time, a TSN backbone and a worst-case timing analysis

ifupauline · version 3.0, November 2026 · code: Apache-2.0 · text: CC BY 4.0

The tool at a glance

You describe the **software** (an application file: task-chains of periodic tasks) and the **hardware** (a configuration file: ECUs, cores, network, and the models and method to use). `mapping` places every task on a core, sets the priorities, analyses the timing of every core, every network port and every task-chain, and tells you how many cores each ECU needs and whether every function meets its deadline.



How many cores?
per ECU, with a lower bound and the margin left (critical scaling factor)

On time?
every task-chain: maximum reaction time against its deadline, why late

Which architecture?
zonal, domain, hybrid or distributed; star, tree, ring or any backbone

Contents

1 What the tool does **4**

2 Installation **5**

3 Concepts in ten minutes **6**

3.1 Task-chains, tasks and their timing . . . 6
3.2 Sub-chains, cores and the utilization cap . . . 6
3.3 ECUs and their roles . . . 7
3.4 Communication: LET and the network . . . 7
3.5 What the analysis gives . . . 8
3.6 The method . . . 8

4 The application file **9**

5 The configuration file **10**

5.1 `platform`: the ECUs . . . 10
5.2 `execution_model`: scheduling and communication . . . 10
5.3 `interference_model`: shared-memory contention . . . 10
5.4 `network_model`: the TSN backbone . . . 12
5.5 `method`: how to map . . . 13

6 Running `mapping` **14**

7 Reading the results **16**

7.1 What the tool understood . . . 16
7.2 The summary table . . . 17
7.3 Per ECU . . . 18
7.4 Cores and task-chains . . . 19
7.5 When the mapping is infeasible . . . 19
7.6 The result files . . . 20

8 Five use cases **20**

8.1 Use case 1: how many cores does the central computer need? . . . 20
8.2 Use case 2: a fail-operational chassis . . . 22
8.3 Use case 3: camera streams and control loops on one backbone . . . 23
8.4 Use case 4: software updates on frozen hardware . . . 24
8.5 Use case 5: a hybrid platform with a big.LITTLE central computer . . . 25

9 The experiments **26**

9.1 How an experiment runs . . . 26
9.2 Experiments of the method . . . 27
9.3 Experiments of the platform . . . 30
9.4 Experiments of the network . . . 31

9.5 Case studies and assumptions 31
9.6 The CSV files 32
9.7 Against other methods: simulated annealing and a genetic algorithm 33
9.8 The exact solver 34
9.9 Checking the bounds by simulation 34

10 Verification **36**

11 Reproducing the paper **36**

12 The code **37**

13 Questions **38**

1 What the tool does

A vehicle function is a *task-chain*: a sensor is sampled, data flows through a sequence of periodic tasks, and an actuator is commanded. A modern electrical/electronic (E/E) platform runs hundreds of them on a few powerful ECUs linked by a Time-Sensitive Networking (TSN) Ethernet backbone. `mapping` takes such an application and such a platform, and answers three questions with a worst-case analysis behind every number:

1. **How many cores** does each ECU need? The tool reports the cores used, a lower bound that no mapping can beat, and the cores it had to add to the platform.
2. **Does every task-chain meet its deadline**, once its tasks are spread over cores and ECUs? The tool bounds the response time of every task (fixed-priority scheduling), the delay of every message (Network Calculus for the TSN shapers) and the maximum reaction time of every task-chain under logical execution time (LET) communication.
3. **How much margin is left?** The critical scaling factor says how much every WCET may grow before a core misses a deadline; the sizing option lowers the utilization cap until a target margin is reached; and the fixed-hardware option checks whether a software update still fits the platform frozen at the start of production.

The method is the one of the paper *System-level mapping of automotive applications onto zonal E/E architectures* (`paper/ifupauline2026-system-level-mapping-automotive.pdf`). The paper explains *why* it works; this manual explains *how* to use it. You need no knowledge of the code before Section 12.

The programs. The build produces six programs; the repository also has five Python scripts.

program	what it is for
`mapping`	map one application on one platform, from files or generated; print or write the results (Sections 6 to 8)
`experiment`	run an experiment of the paper: many design points on many applications, in parallel, to a CSV file (Section 9)
`baselines`	the heuristic against simulated annealing and a genetic algorithm (Section 9.7)
`export_instance`	write an instance for the exact solver, or check an allocation (Section 9.8)
`net_export`	write the flows of a mapping for the network simulator (Section 9.9)
`golden`	regenerate the reference results of the regression test (Section 10)
`exact/exact.py`, `gap.py`	exact allocation with OR-Tools, and the optimality-gap study
`tests/netsim/netsim.py`	frame-level simulation of the TSN network
`tests/syssim/syssim.py`	simulation of the whole system: cores, LET and network
`paper/make_figures.py`	the figures of the paper from the CSV files

Words. The tool, the files and the paper use the words of the field (Table 1). A few names in the code are older or shorter: an ECU is a *controller* in the C sources, a sub-chain a *fragment*, and the central computer of the built-in zonal platform is called `PCU`.

Table 1. The words of the tool.

word	in the files and output	meaning
ECU	`ecus`, `ecu`	electronic control unit: a computer of the platform
central computer	`"role": "central"`	ECU that runs most functions; free software goes there first
zone controller	`"role": "zone"`	ECU of a physical zone, wired to its sensors and actuators
domain controller	`"role": "domain"`	ECU of one vehicle domain (chassis, body, ADAS…)
core type	`big`, `little`, `mcu`, `lockstep`	application cores, energy-efficient cores, microcontroller cores, lock-step safety cores
reference core	`wcet_us`	the WCETs of the application file are on a core of speed 100 %
task-chain	`chains`	sequence of tasks from a sensor to an actuator
sub-chain	*chain.index*, “fragment”	run of consecutive tasks of one chain placed together on one core
utilization cap φ	`utilization_cap`, `--phi`	largest load of a core, its LET task included
sub-chain limit σ	`sigma`, `--sigma`	largest utilization of a sub-chain when chains are cut
allocation rule	`allocation`, `--algorithm`	BFDU, WFDU, FFDU or CADU (Section 5.5)
LET	`"communication": "implicit"`	logical execution time: data cross cores only at fixed instants
maximum reaction time	“reaction”	longest time from an event to the actuation that accounts for it: compared with the deadline
critical scaling factor	“scaling”	largest factor on every WCET that keeps the cores schedulable

2 Installation

Requirements. A C11 compiler (GCC or Clang), CMake 3.10 or later, and OpenMP (optional: experiments in parallel). For the scripts: Python 3 with `numpy`, `matplotlib` and `ortools` (the exact solver only). For the paper and this manual: a LaTeX distribution (on Debian: `texlive-latex-extra texlive-science texlive-pictures texlive-fonts-extra`).

Build and test.

```
cmake -S . -B build
cmake --build build -j
cd build && ctest --output-on-failure
pip install -r requirements.txt   # numpy, matplotlib, ortools
```

The programs are in `build/bin`. All tests must pass (Section 10); the exact-solver test runs only if OR-Tools is installed. A build with the address and undefined-behaviour sanitizers checks the memory accesses of the C code:

```
cmake -S . -B build-san -DMP_SANITIZE=ON -DCMAKE_BUILD_TYPE=Debug
```

First runs. Each command below takes well under a second.

```
build/bin/mapping   # a generated application, built-in zonal platform
build/bin/mapping examples/uc1_central_sizing/application.json \
examples/uc1_central_sizing/configuration.json --cores
build/bin/mapping --example-config domain > domain.json   # a template
build/bin/mapping --config domain.json --chains 120 --out results-domain
```

The first run generates 300 task-chains and compares the four allocation rules on the built-in zonal platform. The second maps the 24 functions of use case 1 (Section 8.1) and prints every core and every task-chain. The last two write a template configuration file and map a generated application on it, the results going to the files of Section 7.6.

3 Concepts in ten minutes

3.1 Task-chains, tasks and their timing

A **task** is periodic: it is released every period T (in microseconds), runs for at most its worst-case execution time (WCET) C, and must end before its next release (implicit deadline). The WCETs of the application file are measured on a *reference core* of speed 100 %; on a core of speed s % a task takes $C \cdot 100/s$, times an optional factor for the type of the core (a neural network may gain more than the speed ratio on a big core). Its **utilization** is C/T: the share of a core it uses.

A **task-chain** is a sequence of tasks, each one using the output of the previous one. Its first task usually reads a sensor and its last one commands an actuator: those tasks are *bound* to the ECU the sensor or the actuator is wired to. The other tasks are *free*: the tool decides where they run. Consecutive tasks with the same period form an *activation pattern*; when the period changes along the chain, the next task reads the last value written, up to one of its periods later.

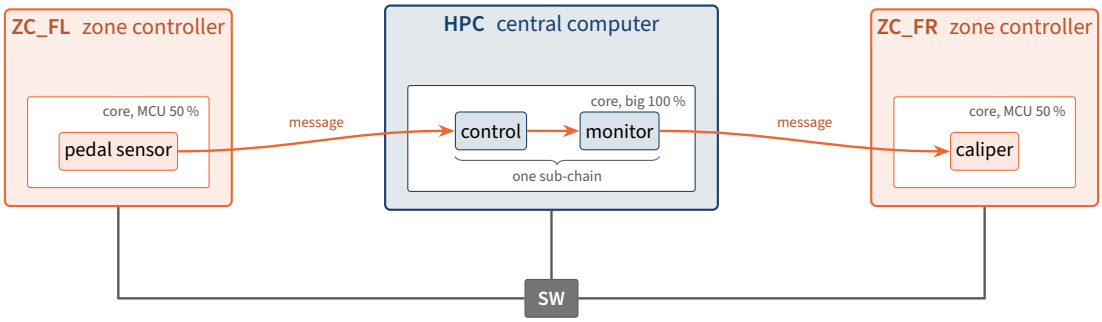


Figure 1. A task-chain of four tasks on a zonal platform: the sensor task is bound to the zone controller of the pedal, the actuator task to the one of the caliper, the two free tasks form one sub-chain on a core of the central computer. Two messages cross the network.

3.2 Sub-chains, cores and the utilization cap

The tool first cuts every task-chain into **sub-chains**: runs of consecutive tasks that will share a core. It cuts where two tasks are bound to different ECUs (*forced cuts*), where they allow different ECUs or core types, and wherever a sub-chain would exceed the **limit** σ (a fraction of a core). Small sub-chains pack better; large ones communicate less.

Every core runs its sub-chains and one **LET task**, at the highest priority, that copies the data between cores. The load of a core is the sum of the utilizations of its tasks and of its LET task; it may not exceed

the **utilization cap** φ (0.8 by default). A core is **schedulable** when every task meets its deadline under fixed priorities; the tool assigns the priorities with an optimal method (Audsley's) restricted to the orders that respect the chain order.

3.3 ECUs and their roles

A platform is a list of ECUs, each with a **role**, its initial cores (by type and speed), the type of core the tool may add, its memory for the tasks and its largest number of cores.

- `central`: a central (vehicle) computer. Free sub-chains go to the central computers first; the cores the tool adds for free sub-chains are added there first. On a platform without a central computer, the ECUs with the fastest cores play this part (the ADAS and infotainment controllers of the domain template).
- `zone`: a zone controller, wired to the sensors and actuators of its area of the car.
- `domain`: a domain controller (powertrain, chassis, ADAS…).
- `ecu`: one ECU of a distributed architecture.

The generator of applications wires the sensors and actuators to every ECU that is not a central computer; in an application file you bind each task yourself.

3.4 Communication: LET and the network

Two consecutive tasks on one core communicate through a register. Between two cores, data cross only at the **LET instants** of the cores: the LET task of the producer core copies the output out, and the LET task of the consumer core reads it at the release of the consumer job (Figure 2). Between two ECUs the copy also crosses the network. The key `communication` of the `execution_model` chooses when an output is copied out:

- `implicit` (the default): at the first LET instant after the producer ends, as in Figure 2. The delays are short, but they depend on the response time of the producer.
- `LET`: **strict LET**, at the end of the period of the producer, whatever its response time. The data flow is deterministic and the messages leave with almost no jitter, but every crossing waits a full producer period. Option `--strict-let`.

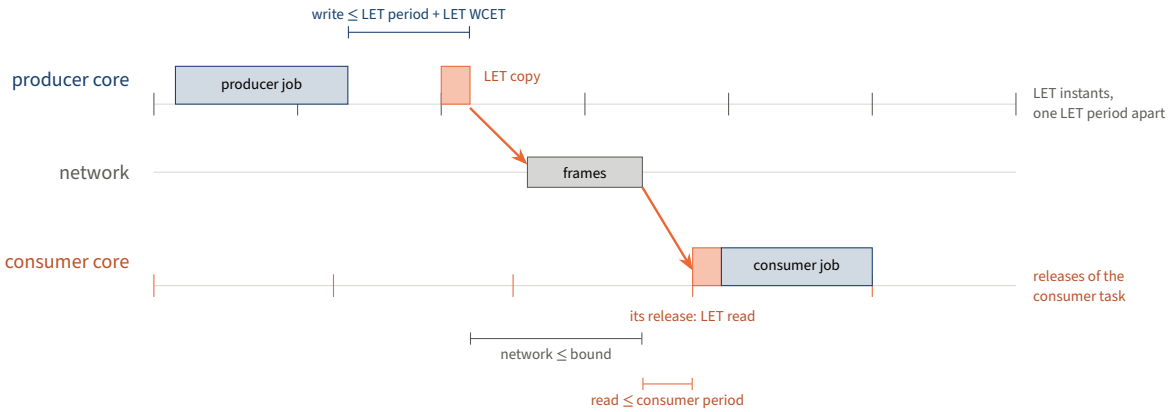


Figure 2. One message between two cores, with implicit communication: the output waits for the next LET instant of the producer core and for the copy (with strict LET, for the end of the producer's period); it crosses the network (between two ECUs); it is read at the next release of the consumer. The analysis bounds each delay.

The network is a switched Ethernet with the TSN mechanisms of IEEE 802.1: credit-based shaping (**CBS**, 802.1Qav), time-aware shaping (**TAS**, 802.1Qbv), asynchronous traffic shaping (**ATS**, 802.1Qcr), frame preemption (802.1Qbu/802.3br) and frame replication and elimination for reliability (**FRER**, 802.1CB). The messages are grouped in one to three *traffic classes* by the period of their producer. Network Calculus bounds the delay of every message at every output port of its route.

3.5 What the analysis gives

- **Response time** of a task: the longest time from its release to its end, with the interference of the higher-priority tasks of its core (and of the shared memory, with the interference model).
- **End-to-end response time** of a chain: the holistic sum of the response times of its pattern runs and of the network bounds, without any copy or sampling delay. It is a figure of merit, not a time of the system.
- **End-to-end delay**: from the release of the job that samples the sensor to the end of the first actuator job that uses the sample, with every sampling and LET delay.
- **Maximum reaction time**: the end-to-end delay plus one period of the first task (an event waits at most that long to be sampled). It is compared with the **deadline** of the chain. It also bounds the maximum data age.
- **Critical scaling factor**: the largest factor by which every WCET of a core may be multiplied with its tasks still on time; the smallest over the cores is the margin of the platform. It is below 1 when a core misses a deadline.

3.6 The method

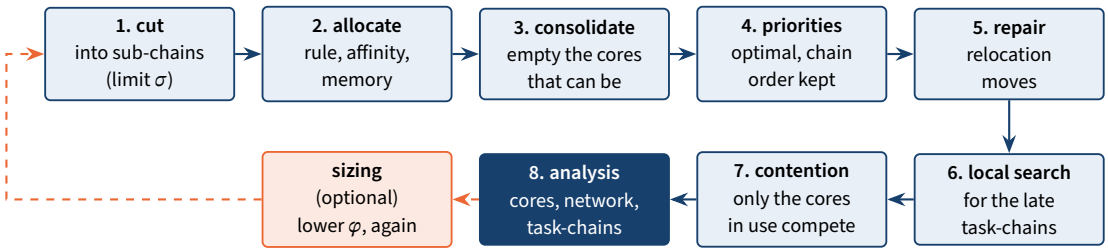


Figure 3. The steps of one design point. With a target margin, the sizing runs the steps again with a lower cap until the critical scaling factor reaches the target.

Figure 3 shows the steps of one *design point* (one mapping of one application with one set of parameters). Step 2 places the sub-chains by decreasing utilization with one of four rules: best fit (BFDU), worst fit (WFDU), first fit (FFDU) or communication aware (CADU, fewest network hops created first); it tries first the cores that already run part of the same chain (*affinity*), and adds a core when no core can take a sub-chain. Step 3 empties the cores whose sub-chains fit elsewhere. Steps 5 and 6 move sub-chains to make every core schedulable and to bring the late chains on time. The paper gives every step in pseudo-code.

4 The application file

The application file is a JSON object with an optional `description` and a list of `chains`. Every chain has a list of `tasks`, in chain order. Times are integers in microseconds; ECUs are named as in the configuration file. The smallest file:

```
{"chains": [{"tasks": [{"period_us": 10000, "wcet_us": 500}]}]}
```

A realistic chain, from the example of `--example-application`:

```
{"name": "brake_by_wire", "deadline_us": 10000, "tasks": [
{"name": "pedal_sensor", "period_us": 1000, "wcet_us": 40,
 "ecu": "ZCU1", "message_bytes": 16},
{"name": "brake_control", "period_us": 1000, "wcet_us": 120,
 "message_bytes": 32},
{"name": "caliper_front_left", "period_us": 1000, "wcet_us": 30,
 "ecu": "ZCU2"}]}
```

The sensor task is bound to `ZCU1`, the actuator task to `ZCU2`; the control task is free. The chain must react within 10 ms.

Table 2. The keys of a task-chain.

key	value	meaning
`name`	text	shown in the outputs; at most 31 characters; distinct names (default: the chain number)
`deadline_us`	integer	end-to-end deadline on the maximum reaction time; absent or 0: none
`ecu`	ECU name	the whole chain on that ECU (a local chain: every task bound there)
`ecus`	ECU names	the ECUs every task of the chain may run on
`replica_group`	integer > 0	chains of one group are redundant copies: never on the same ECU (or core, Section 5.5); a group has at least two chains
`tasks`	list	the tasks, in chain order (mandatory)

Table 3. The keys of a task. Only the period and the WCET are mandatory.

key	value	meaning
`name`	text	shown in `mapping.csv`; at most 31 characters
`period_us`	integer ≥ 1	period, also the deadline of the task
`wcet_us`	integer ≥ 1	WCET on the reference core (speed 100 %), at most the period
`ecu`	ECU name	the task is bound there (its sensor or actuator)
`ecus`	ECU names	the ECUs it may run on (not with `ecu`)
`core_types`	list	the core types it may run on: `big`, `little`, `mcu`, `lockstep` (a safety task: `["lockstep"]`)
`wcet_factor_percent`	{type: %}	WCET on a core of that type, in % of the WCET at the same speed (e.g. `{"big": 70}`: 30 % faster than its speed ratio on big cores)
`memory_kb`	integer	code and data, counted against the memory of its ECU
`message_bytes`	integer	size of its output message (absent: one minimum-size frame)
`memory_accesses`	integer	requests to the shared memory per job (interference model; absent: from the access density)
`colocate_with_next`	true/false	must share a core with the next task

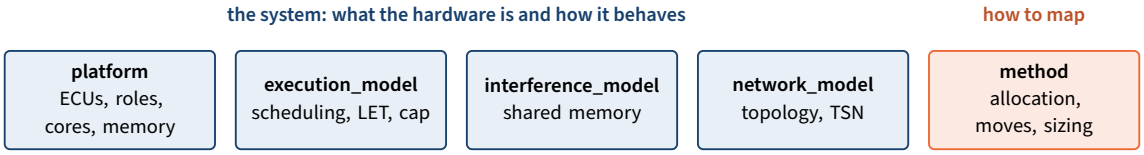
Checks. The file is checked strictly: an unknown key, a value of the wrong type or out of range, an unknown ECU, a WCET above its period, a task bound to another ECU than its chain, a replica group of one chain or two chains with the same name are errors, reported with the file, the line and the key:

```
app.json:14:  chain brake, task pedal:  no ECU called "ZCU9"
```

Generated applications. `mapping --generate --chains 200 --seed 4 > app.json` writes a generated application (the statistics of an industrial engine-management software, see the appendix of the paper) for the platform of the configuration given with `--config`; you can then edit it. The options `--replicas`, `--zone-only`, `--colocate`, `--lockstep` and `--type-variation` add constraints to the generated chains.

5 The configuration file

The configuration file describes the hardware and the way to map onto it. It has an optional `description` and five sections, each optional: what a section leaves out keeps the default value of Tables 4 to 8. Without a `platform` section the platform is the built-in zonal one.



Start from a template. `mapping --example-config T` prints a complete configuration file, every key with its value, for the template `T`: `zonal` (the built-in platform), `domain`, `hybrid` or `distributed` (Figure 4). `mapping --config FILE --print-config` prints the configuration in effect, options included: the way to check what the tool understood.

5.1 `platform`: the ECUs

`"platform": {"ecus": [...]}` lists 1 to 32 ECUs. The ECU order sets the numbering of the cores (central computers first, then one core of each ECU of a role at a time) and the order in which ECUs receive new cores.

```
{"name": "HPC", "role": "central",
 "cores": [{"type": "big", "count": 4, "speed_percent": 100},
{"type": "little", "count": 4, "speed_percent": 40}],
 "added_core_type": "big", "memory_kb": 524288, "max_cores": 12}
```

5.2 `execution_model`: scheduling and communication

5.3 `interference_model`: shared-memory contention

The cores of one ECU share its memory; a request waits for the requests of the other cores, served in round robin. A task then runs for its WCET plus its requests times $(n-1)$ times the latency, n the cores of the ECU that run tasks; the tool iterates until the number of competing cores is the one of the mapping.

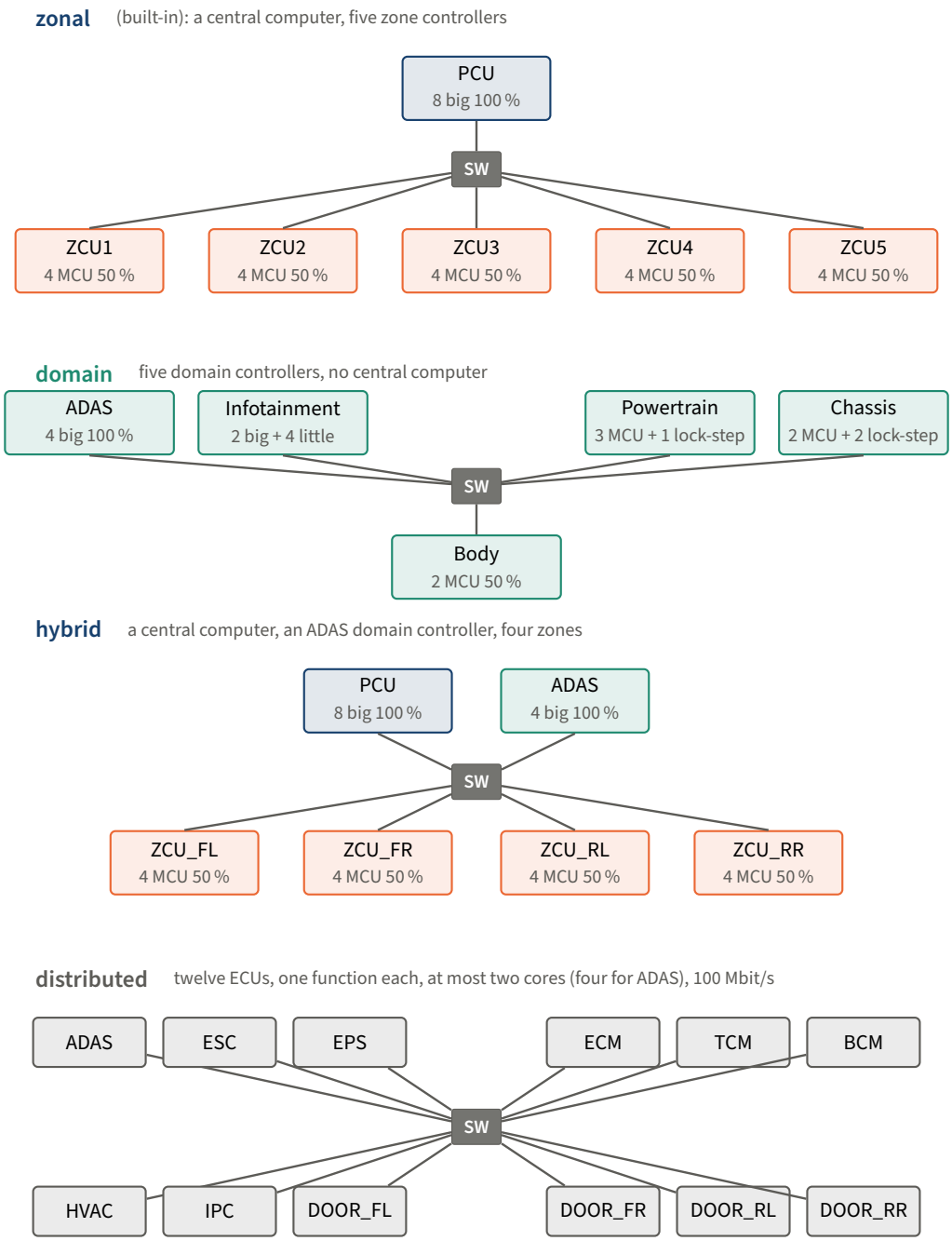


Figure 4. The four templates of `--example-config`, star topology. The zone, domain and distributed ECUs are wired to the sensors and actuators of generated applications; ESC and EPS have one lock-step core each, ADAS two big cores, the others one microcontroller core at 50 %.

Table 4. The keys of an ECU (`platform.ecus`).

key	default	meaning
`name`	(mandatory)	at most 31 characters, distinct
`role`	(mandatory)	`central`, `domain`, `zone` or `ecu` (Section 3)
`cores`	(mandatory)	one group per core type: `type` (`big`, `little`, `mcu`, `lockstep`), `count` (initial cores, may be 0) and `speed_percent` (of the reference core). A type without a group does not exist on this ECU
`added_core_type`	the first type	type of the cores the tool adds to this ECU (it adds a core of another type of the ECU when a sub-chain may only run there)
`memory_kb`	0 (unlimited)	memory for the code and data of the tasks
`max_cores`	0 (unlimited)	largest number of cores: a fixed hardware variant

Table 5. The keys of `execution_model`.

key	default	meaning
`scheduling`	`fixed-priority preemptive`	the only one supported; written for the reader of the file
`communication`	`implicit`	between cores: `implicit` (an output is copied at the first LET instant after its producer ends) or `LET` (strict LET: published at the end of its producer's period) (Section 3.4)
`utilization_cap`	0.8	φ: largest load of a core, LET task included
`let_task_wcet_us`	50	WCET of the LET task of a core that communicates, on the reference core
`wcet_margin_percent`	0	every WCET is raised by this percentage before the mapping

5.4 `network_model`: the TSN backbone

Every ECU has one full-duplex link to a switch. The `topology` gives the switches: `star` (one switch), `tree` (one switch per ECU, all linked to the switch of the root ECU), `ring` (one switch per ECU, in ECU order) or `custom` (Figure 5). Routes are shortest paths; the links of the central computers run at the backbone rate.

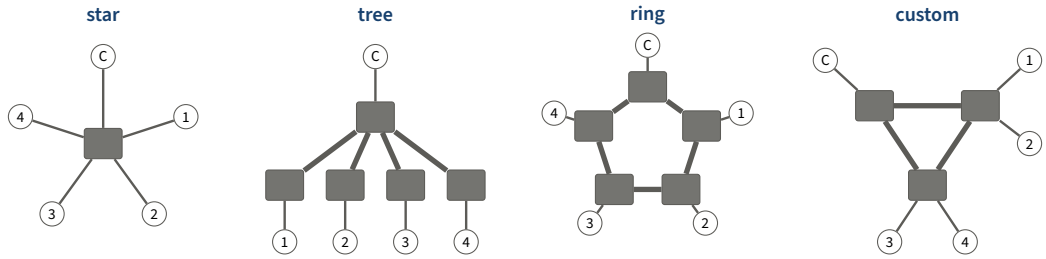


Figure 5. The topologies, for a central computer C and four other ECUs: the switches are dark, the backbone links thick. A custom topology may have switches without ECUs and links of their own rates.

A custom topology names its switches with the ECUs on each one, and the links between switches (full duplex, at the backbone rate unless they give theirs):

```
"topology": "custom",
"switches": [{"name": "SW_CENTRAL", "ecus": ["HPC"]},
{"name": "SW_FRONT", "ecus": ["ADAS_DC", "ZC_FL", "ZC_FR"]},
{"name": "SW_REAR", "ecus": ["ZC_RL", "ZC_RR"]}],
"switch_links": [{"from": "SW_CENTRAL", "to": "SW_FRONT", "mbps": 10000},
{"from": "SW_CENTRAL", "to": "SW_REAR", "mbps": 2500},
```

Table 6. The keys of `interference_model`.

key	default	meaning
`shared_memory_arbitration`	`round-robin`	the only one supported
`memory_latency_ns`	0	time to serve one request; 0: no contention
`memory_accesses_per_ms`	1000	requests per ms of reference WCET, for the tasks without `memory_accesses` and for the LET tasks

```
{"from":  "SW_FRONT", "to":  "SW_REAR"}]
```

The tool checks that every ECU is on one switch, that every switch can reach every other one, and that no route is longer than 16 ports. With FRER (ring or custom), the second route of a replicated message is the shortest one that shares no link and no switch with the first between the switch of the source and the switch of the destination.

Table 7. The keys of `network_model`.

key	default	meaning
`standard`	`TSN`	the only one supported
`topology`	`star`	`star`, `tree`, `ring` or `custom`
`tree_root`	first central	tree: the ECU whose switch is the root
`switches`, `switch_links`		custom topology only (above)
`link_mbps`	1000	rate of the ECU links, Mbit/s
`backbone_mbps`	link rate	rate of the links between switches and of the links of the central computers
`ecu_link_mbps`		rates of single ECU links, e.g. `{"HPC":  10000}`
`switch_latency_us`	2	fabric and propagation delay per switch crossed
`link_limit_percent`	100	design limit of the LET load of a port; a port above it is overloaded
`traffic_shaping`	`CBS`	`CBS`, `TAS` or `ATS`
`traffic_classes`	1	1 to 3 LET classes, by the period of their producer
`class_max_periods_us`	5000, 20000	largest producer period of each class but the last (classes −1 values)
`idle_slopes_percent`	75 (one class)	CBS: share of the port rate of each class
`proportional_idle_slopes`	true	CBS, several classes: the total of the shares divided at each port in proportion to the traffic of each class
`tas_cycle_us`	500	TAS: gate cycle
`tas_windows_percent`	70 (one class)	TAS: window of each class, % of the cycle
`frame_preemption`	false	preemption of the best-effort frames
`frer`	false	ring or custom: messages of replicated chains over two disjoint routes

5.5 `method`: how to map

Which rule?. FFDU and BFDU give the fewest cores; CADU the fewest network messages, at the price of more cores; WFDU spreads the load on every core, which is wasteful at design time but good on hardware that is already paid for (use case 4, Section 8.4). `--algorithm all` compares the four.

Table 8. The keys of `method`.

key	default	meaning
`allocation`	`FFDU`	BFDU (best fit: fullest core), WFDU (worst fit: emptiest core), FFDU (first fit), CADU (fewest network hops, then a central computer, then best fit); all by decreasing utilization
`sigma`	0.4	σ: largest utilization of a sub-chain, fraction of a reference core (at most the cap)
`affinity`	true	a sub-chain first tries the cores of the other sub-chains of its chain
`consolidation`	true	after the allocation, empty the cores whose sub-chains fit elsewhere (not with WFDU)
`swap_moves`	false	also exchange sub-chains of two unschedulable cores, before the relocation moves (in the experiments it never repaired a core)
`relocation_moves`	true	move sub-chains of unschedulable cores to schedulable ones
`local_search`	true	move sub-chains to bring late task-chains on time
`replica_separation`	`ecu`	replicas on different `ecu`s, or only on different `core`s
`fixed_hardware`	false	add no core: a sub-chain that fits nowhere makes the mapping infeasible (software updates of a frozen platform)
`target_scaling`	0 (none)	sizing: lower the cap until the critical scaling factor reaches this value (e.g. 1.3)

6 Running `mapping`

```
mapping [APPLICATION.json [CONFIGURATION.json]] [options]
```

- **The application**: the file given first, or else an application generated from `--seed` and `--chains` (or the small detection chain with `--detection-chain`).
- **The configuration**: the file given second (or with `--config`), or else the built-in zonal platform and the default method.
- **The rules**: without a configuration file the four allocation rules are compared side by side; with one, the tool uses the rule of its `method` section. `--algorithm A` chooses one rule, `--algorithm all` the four.
- **Options override the configuration file.** The options of the built-in zonal platform (`--pcu-*`, `--zcu-*`, `--little-speed`) are refused when the file has its own `platform`: change the file instead.

Exit status. 0 when the tool ran (whether the mapping is feasible or not), 1 on an internal error, 2 on an invalid option or file; the error message gives the file, the line and the key.

Table 9. The options of `mapping` (`mapping --help`). The configuration key each one overrides is in brackets.

option	meaning
files	
`--config FILE`	the configuration file (instead of the second argument)
`--example-config [T]`	print the template `zonal`, `domain`, `hybrid` or `distributed`, then exit
`--example-application`	print a small application file for the zonal platform, then exit
`--generate`	print the generated application as a file, then exit
`--print-config`	print the configuration in effect, options included, then exit
`--out DIR`	write the result files of Section 7.6 into `DIR`
generated application	
`--seed N`, `--chains N`	seed (default `$MP_SEED` or 1) and number of task-chains (300)
`--detection-chain`	the detection chain: an illustrative chain of nine tasks
`--replicas P`	% of the chains that are replicas of others (0)
`--colocate P`	% of consecutive free tasks that must share a core (0)
`--zone-only P`	% of chains whose free tasks avoid the central computers (0)
`--lockstep P`	% of chains (safety) on lock-step cores only (0)
`--type-variation P`	WCET on each core type within $\pm P$ % of its speed ratio (0)
method	
`--algorithm A`	`BFDU`, `WFDU`, `FFDU`, `CADU` or `all` [`allocation`]
`--sigma S`	sub-chain limit, fraction of a core [`sigma`]
`--target-scaling F`	sizing for a critical scaling factor F [`target_scaling`]
`--no-affinity`, `--no-consolidation`	[`affinity`], [`consolidation`] off
`--no-relocation`	[`relocation_moves`] off
`--swap`	[`swap_moves`] on
`--no-repair`	no deadline-driven local search [`local_search`]
`--fixed-hardware`	add no core [`fixed_hardware`]
`--separate-cores`	replicas on different cores only [`replica_separation`]
execution and interference models	
`--phi F`	utilization cap [`utilization_cap`]
`--let C`	WCET of the LET tasks, µs [`let_task_wcet_us`]
`--strict-let`	strict LET between cores instead of implicit communication [`communication`]
`--margin P`	WCET margin, % [`wcet_margin_percent`]
`--memory-latency L`	ns per memory request [`memory_latency_ns`]
`--access-density N`	requests per ms of WCET [`memory_accesses_per_ms`]
network model	
`--topology T`	`star`, `tree` or `ring` [`topology`]
`--link M`, `--backbone M`	rates, Mbit/s [`link_mbps`, `backbone_mbps`]
`--classes N`	1 to 3 traffic classes, with their default boundaries and shares [`traffic_classes`]
`--tas`, `--ats`	time-aware or asynchronous shaping [`traffic_shaping`]
`--preemption`, `--frer`	[`frame_preemption`], [`frer`]
built-in zonal platform only	
`--pcu-cores N`, `--pcu-speed P`	big cores of the central computer (8) and their speed (100 %)
`--pcu-little N`, `--little-speed P`	little cores of the central computer (0) and their speed (40 %)
`--zcu-cores N`, `--zcu-speed P`	cores of each zone controller (4) and their speed (50 %)
`--zcu-lockstep N`	lock-step cores of each zone controller (0)
`--pcu-memory K`, `--zcu-memory K`	memory for the tasks, KB (unlimited)
output	
`--cores`	also print every core and every task-chain

7 Reading the results

This section reads the output of use case 1 (Section 8.1), produced by

```
mapping examples/uc1_central_sizing/application.json \
examples/uc1_central_sizing/configuration.json --cores
```

7.1 What the tool understood

The output starts with the application and the platform as the tool read them: the number of task-chains and tasks, how many are bound to one ECU, how many start with a sensor task and end with an actuator task, every ECU with its cores, then the models and the method.

```
examples/uc1_central_sizing/application.json: 24 task-chains, 73 tasks
3 local to one ECU; of the others, 19 start at a bound sensor task, 19 end at a bound actuator task
platform: 6 ECUs
  HPC          central  2 big at 100%, 65536 KB, at most 16 cores
  ZC_FL        zone     2 MCU at 50%, 4096 KB, at most 4 cores
  ZC_FR        zone     2 MCU at 50%, 4096 KB, at most 4 cores
  ZC_RL        zone     2 MCU at 50%, 4096 KB, at most 4 cores
  ZC_RR        zone     2 MCU at 50%, 4096 KB, at most 4 cores
  ZC_C         zone     2 MCU at 50%, 4096 KB, at most 4 cores
execution: fixed-priority preemptive, implicit communication at LET instants, utilization cap 0.80, LET task 50 us, WCET margin 0%
network: star, 1000 Mbit/s (backbone 1000), 1 CBS class
```

Table 10. The rows of the summary table.

row	meaning
feasible	every sub-chain found a core; otherwise the line below the table says which one did not, and why (Section 7.5)
sub-chains, fragmentation rate, forced cuts	the partitioning: sub-chains, cuts in % of the possible cuts, cuts between tasks bound to different ECUs
cores used M	cores that run at least one task: **the sizing result**; then on central computers and on the other ECUs
lower bound on M	no mapping of these sub-chains uses fewer cores (each ECU's bound load, and the total load, over the cap)
cores added, cores emptied	cores beyond the initial ones; cores emptied by the consolidation
schedulable cores SR	% of cores where every task meets its deadline, after the priorities, the optional swaps, the relocations, and at the end
critical scaling factor	the margin of the platform (Section 3.5); with sizing, the cap found and whether the target was reached
application / LET load, room left	in cores, summed over the cores used
task-chains schedulable / meeting deadline	% of chains whose tasks are all schedulable; and also within their deadline over bounded routes
local search moves	moves kept by the deadline-driven local search
communications inside an ECU / inter-ECU	messages between two cores of one ECU (shared memory) and between ECUs (network flows)
replicated with FRER	flows sent over two routes
network delay bound, busiest port	the largest bound of a flow; the largest LET load of an output port
end-to-end response time, max reaction time	the largest values over the task-chains (Section 3.5)
CPU time	time to compute the design point (and with the sizing)

7.2 The summary table

One column per allocation rule (here only FFDU, the rule of the file).

metric	FFDU
feasible	yes
sub-chains	64
fragmentation rate FR (%)	81.6
cuts forced by the ECUs	40
cores used M	7
on central computers	2
on the other ECUs	5
lower bound on M	5
cores added to the architecture	0
cores emptied by consolidation	0
schedulable cores SR (%)	100.0
after scheduling	100.0
after swap moves	100.0
after relocation moves	100.0
critical scaling factor	1.302
application load (cores)	2.70
LET load (cores)	0.34
room left below the cap (cores)	2.56
task-chains schedulable (%)	100.0
task-chains meeting deadline (%)	100.0
deadline misses	0
local search moves	6
communications inside an ECU	0
inter-ECU communications	33
replicated with FRER	0
network delay bound, max (us)	786.8
busiest port load (%)	0.53
end-to-end response time, max (ms)	61.38
max reaction time, max (ms)	2007.44
CPU time (ms)	0.7

Partitioning. 64 sub-chains for 73 tasks: most chains are cut at both ends, where the sensor and the actuator are bound to zone controllers (40 forced cuts).
Cores. 7 cores are used: 2 on the HPC and one in each zone controller. The lower bound is 5: no mapping of these sub-chains can do with fewer. No core was added; none had to be emptied.
Schedulability. Every core is schedulable after the priority assignment; the relocation moves had nothing to do. The critical scaling factor 1.302: every WCET may grow by 30 % before a core misses a deadline.
Load. 2.70 cores of application load, 0.34 of LET copies, 2.56 cores left below the caps.
Task-chains. All on time; the local search kept 6 moves to get there. 33 messages cross the network; the largest delay bound is 0.8 ms on a port loaded at 0.5 %.
Time. The design point took less than a millisecond of CPU.

7.3 Per ECU

The second table gives, for every ECU and rule, the cores used, their load in cores (LET included) and the smallest critical scaling factor of its cores. Here ZC_FL is the ECU with the least margin: its 1.302 is the factor of the platform.

cores used per ECU, their load (in cores, LET included) and critical scaling factor

ECU	FFDU cores	FFDU load	FFDU scaling
HPC	2	0.91	1.917
ZC_FL	1	0.78	1.302
ZC_FR	1	0.67	1.573
ZC_RL	1	0.43	2.768
ZC_RR	1	0.15	7.059
ZC_C	1	0.10	12.438

7.4 Cores and task-chains

With `--cores`, a table per rule lists the cores in use: ECU, type and speed, load and LET load (% of the core), scaling factor, schedulability, number of tasks, and the sub-chains on it as *chain.index*.

FFDU: cores in use

core	ECU	type speed %	load %	LET %	scaling factor	sched	tasks	sub-chains chain.index
c0	HPC	big 100	39.0	1.0	2.604	yes	7	6.1 3.1 23.0 16.1 14.1
c1	HPC	big 100	52.4	0.5	1.917	yes	13	5.1 7.1 8.1 21.1 5.2 9.1 10.1 22.0 11.1 12.1 13.1
c2	ZC_FL	MCU 50	78.2	10.0	1.302	yes	14	6.0 0.0 2.0 6.2 1.0 7.0 18.0 4.0 9.0 13.2 15.2 0.1
c3	ZC_FR	MCU 50	67.2	10.0	1.573	yes	11	2.2 0.2 5.0 19.0 9.2 6.3 15.0 2.1 15.1
c4	ZC_RL	MCU 50	42.5	10.0	2.768	yes	11	1.2 3.2 21.0 14.2 16.2 11.2 20.0 17.0 1.1 17.1
c5	ZC_RR	MCU 50	15.0	1.0	7.059	yes	6	10.0 5.3 4.2 11.0 10.2 4.1
c6	ZC_C	MCU 50	9.9	2.0	12.438	yes	11	8.0 3.0 21.2 7.2 13.0 14.0 16.0 8.2 12.0 12.2 17.2

A second table follows every task-chain: its path as segments on cores, with the indices of its sub-chains in brackets; the number of communications inside an ECU and over the network; its maximum reaction time and its deadline; and, if it is late, why: *task late* (a task misses its period), *network* (a flow crosses an overloaded port) or *deadline* (the reaction time exceeds the deadline).

FFDU: task-chains

chain	tasks	load %	path: segments on cores (sub-chains)	comm. ECU/net	reaction ms	deadline ms	on time	why late
brake_by_wire_front	3	21.00	c2@ZC_FL(0-1) > c3@ZC_FR(2)	0/1	4.5	5.0	yes	
brake_by_wire_rear	3	17.00	c2@ZC_FL(0) > c4@ZC_RL(1-2)	0/1	4.1	5.0	yes	
steer_by_wire	3	25.00	c2@ZC_FL(0) > c3@ZC_FR(1-2)	0/1	4.3	5.0	yes	
stability_control	4	15.80	c6@ZC_C(0) > c0@HPC(1) > c4@ZC_RL(2)	0/2	27.2	30.0	yes	
propulsion_torque	3	4.70	c2@ZC_FL(0) > c5@ZC_RR(1-2)	0/1	23.4	30.0	yes	
adaptive_cruise	4	18.10	c3@ZC_FR(0) > c1@HPC(1-2) > c5@ZC_RR(3)	0/2	138.4	200.0	yes	
lane_keeping	4	30.90	c2@ZC_FL(0) > c0@HPC(1) > c2@ZC_FL(2) > c3@ZC_FR(3)	0/3	147.8	150.0	yes	
traffic_sign_recognition	3	13.60	c2@ZC_FL(0) > c1@HPC(1) > c6@ZC_C(2)	0/2	365.1	400.0	yes	
driver_monitoring	3	9.53	c6@ZC_C(0) > c1@HPC(1) > c6@ZC_C(2)	0/2	356.0	400.0	yes	
park_assist	4	3.98	c2@ZC_FL(0) > c1@HPC(1) > c3@ZC_FR(2)	0/2	282.5	300.0	yes	
battery_management	3	4.55	c5@ZC_RR(0) > c1@HPC(1) > c5@ZC_RR(2)	0/2	267.9	400.0	yes	
thermal_management	3	0.83	c5@ZC_RR(0) > c1@HPC(1) > c4@ZC_RL(2)	0/2	563.0	1000.0	yes	
climate_control	3	0.80	c6@ZC_C(0) > c1@HPC(1) > c6@ZC_C(2)	0/2	667.0	1000.0	yes	
headlights	3	1.00	c6@ZC_C(0) > c1@HPC(1) > c2@ZC_FL(2)	0/2	80.1	100.0	yes	
turn_indicators	3	0.55	c6@ZC_C(0) > c0@HPC(1) > c4@ZC_RL(2)	0/2	72.5	100.0	yes	
wipers	3	0.22	c3@ZC_FR(0-1) > c2@ZC_FL(2)	0/1	232.6	300.0	yes	
central_locking	3	0.50	c6@ZC_C(0) > c0@HPC(1) > c4@ZC_RL(2)	0/2	327.0	500.0	yes	
tire_pressure	3	0.04	c4@ZC_RL(0-1) > c6@ZC_C(2)	0/1	2007.4	3000.0	yes	
window_front_left	3	0.90	c2@ZC_FL(0)	0/0	23.9	60.0	yes	
window_front_right	3	0.90	c3@ZC_FR(0)	0/0	21.7	60.0	yes	
seat_heating	2	0.01	c4@ZC_RL(0)	0/0	1002.5	-	yes	
instrument_cluster	3	7.20	c4@ZC_RL(0) > c1@HPC(1) > c6@ZC_C(2)	0/2	74.3	100.0	yes	
energy_management	2	1.90	c1@HPC(0)	0/0	135.4	-	yes	
diagnostics	2	0.80	c0@HPC(0)	0/0	1021.3	-	yes	

Reading a path. `c6@ZC_C(0) > c0@HPC(1) > c4@ZC_RL(2)` (stability control) means: sub-chain 0 runs on core c6 of zone controller `ZC_C`, sub-chain 1 on core c0 of the HPC, sub-chain 2 on core c4 of `ZC_RL`. The two arrows are two inter-ECU communications, each costing LET delays and a network transfer: 27 ms of reaction time for tasks of 5 ms.

7.5 When the mapping is infeasible

A mapping is infeasible when a sub-chain fits on no core. The tool names it and says why:

```
FFDU: infeasible, sub-chain 23.0 (0.8 % of a reference core, 6144 KB,
free):  not enough memory, at most 5120 KB left on the ECUs it may run on
```

The other reasons are: no core of the ECUs it may run on has a type it allows; the hardware is fixed (`--fixed-hardware`); the ECUs with memory left already have their largest number of cores; or the sub-chain is larger than an empty core (lower `--sigma`, or use faster cores).

7.6 The result files

`--out DIR` writes six files, one row per rule and item, so that a spreadsheet or a script can take over.

Table 11. The files of `--out`.

file	content
`summary.json`	one object per rule: the rows of the summary table, as numbers
`ecus.csv`	per rule and ECU: role, cores used, application and LET load (cores), critical scaling factor, memory used and capacity (KB), load of its egress and ingress network ports (%)
`cores.csv`	per rule and core in use: ECU, type, speed, added or initial, load and LET load (%), schedulable, critical scaling factor, tasks
`chains.csv`	per rule and task-chain: tasks, utilization, sub-chains, communications over the network and inside an ECU, end-to-end response time, maximum reaction time (both empty when the chain has no bound: a task above its period or an unbounded port), deadline, schedulable, network bounded, on time, why late
`mapping.csv`	per rule and task: chain and task (numbers and names), sub-chain, core, ECU, core type, priority, period, WCET on its core, response time (empty for a task above its period, which has none)
`system.json`	the timing model of each mapping (cores with their LET task and tasks, chains with the core of each task and the bound of each hop), for the full-system simulation (Section 9.9)

8 Five use cases

Each use case is a realistic question, with its application and configuration files in `examples/`. Together they use every key of the two files. Run them from the root of the repository; the outputs below come from these exact commands.

use case	what it shows
1. sizing the central computer	a zonal platform, 24 vehicle functions; read every output; compare the rules; size for a margin
2. a fail-operational chassis	redundant task-chains, lock-step cores, a ring with frame replication (FRER), CADU
3. camera streams and control loops	a tree backbone, three traffic classes, TAS, ATS, frame preemption, link rates
4. software updates	sizing at the SOP, then updates on frozen hardware; why spreading the load helps then
5. a hybrid platform	big.LITTLE central computer, a domain controller, shared-memory contention, a custom backbone

8.1 Use case 1: how many cores does the central computer need?

The question. A carmaker moves 24 functions of chassis, powertrain, ADAS, body and comfort onto a zonal platform: a high-performance computer (HPC) and five zone controllers. The zone controllers are chosen (two microcontroller cores at half the speed of the reference core, 4 MB for the tasks); the HPC variant is open. How many cores must it have, and is every function on time?

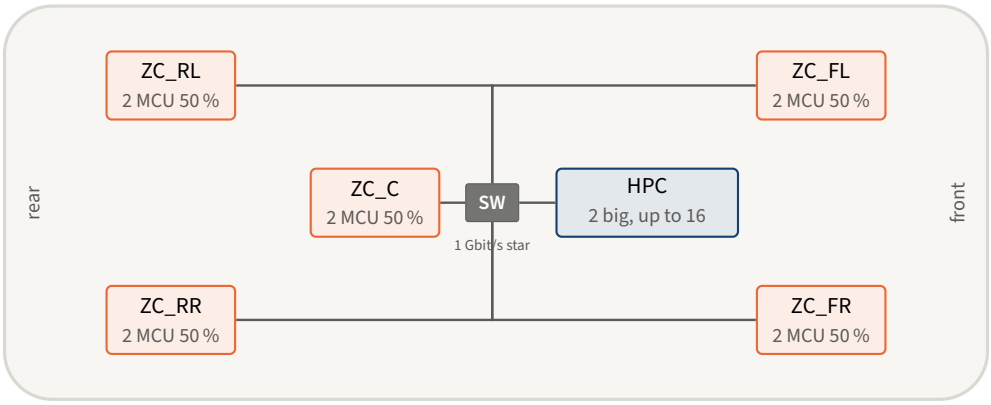


Figure 6. Use case 1: the platform, seen from above (front on the right). Each zone controller has 4 MB and at most 4 cores; the HPC 64 MB and at most 16 cores.

The files. `examples/uc1_central_sizing/`: the configuration lists the six ECUs; the HPC starts with only 2 big cores, so that the tool adds the cores it needs (`max_cores` 16). The application has 24 task-chains and 73 tasks: brake-by-wire and steer-by-wire at 1 ms with 5 ms deadlines, stability and propulsion control at 5 and 10 ms, ADAS perception at 40 to 100 ms (big cores only, up to 16 MB), body and comfort functions at 20 ms to 1 s, and three local chains (two window lifters, the seat heating) bound to one zone controller each.

Run.

```
mapping examples/uc1_central_sizing/application.json \
examples/uc1_central_sizing/configuration.json --cores
```

Section 7 reads this output line by line. The answer: **two HPC cores suffice** (7 cores in all, the lower bound being 5), every core is schedulable with a margin of 30 % on every WCET, and all 24 functions meet their deadlines.

Compare the rules. `--algorithm all` maps with the four rules:

metric	BFDU	WFDU	FFDU	CADU
cores used M	7	12	7	6
critical scaling factor	1.302	1.254	1.302	1.465
task-chains meeting deadline (%)	100.0	95.8	100.0	95.8
inter-ECU communications	33	33	33	22
max reaction time, max (ms)	2007.44	2007.28	2007.44	2019.49

CADU saves a core and a third of the network messages here, but one chain misses its deadline; WFDU spreads the load over 12 cores and also leaves one chain late. The largest maximum reaction time is about the same for all rules: it is the chain of the tire pressure monitor, whose tasks run once a second and whose deadline is 3 s.

Size for a margin. A margin of 30 % on every WCET is little at the start of a project. `--target-scaling` lowers the utilization cap until the critical scaling factor reaches the target:

target	cores	cap found	scaling	on time (%)
none	7	-	1.302	100.0
1.5	7	0.709	1.681	95.8
2	7	0.525	2.008	87.5

The cores are the same, but the load is spread so that every WCET may grow by 68 % (target 1.5) or 100 % (target 2). The price is paid in deadlines: with a lower cap, the control tasks of the 1 ms by-wire chains no longer fit the zone controller of their sensor and move to the HPC, which adds two network hops. With the target 1.5 the steer-by-wire chain is late, with the target 2 the two brake-by-wire chains too; the `--cores` output names them.

8.2 Use case 2: a fail-operational chassis

The question. Steer-by-wire and brake-by-wire must survive the failure of one ECU or one network link. Each is built as two redundant task-chains (two *lanes*) with their own sensors and actuators; their safety monitors must run on lock-step cores; the messages of the redundant chains are sent twice, over both directions of an Ethernet ring (IEEE 802.1CB FRER). Is the platform of Figure 7 enough, and what does the redundancy cost on the network?

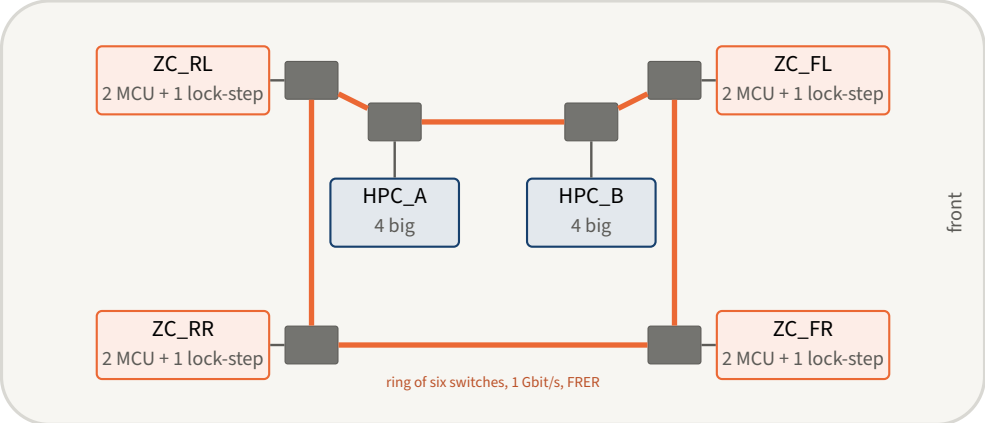


Figure 7. Use case 2: two central computers and four zone controllers with lock-step cores, one switch per ECU, the switches in a ring in the order of the configuration file.

The files. `examples/uc2_redundant_chassis/`. The configuration sets a cap of 0.7 and a WCET margin of 10 % (safety functions), a ring with FRER, two CBS classes (messages of tasks up to 5 ms in the first one, with 50 % of the port, the others with 25 %), and the CADU rule with replicas on different ECUs. In the application, `replica_group` 1 joins the two steering lanes and group 2 the two braking lanes; the monitors have `"core_types": ["lockstep"]` and the ECUs they may use (`ecus`); the airbag is a local chain of the front right zone on lock-step cores; the state estimator and the torque vectoring of the vehicle dynamics share a core (`colocate_with_next`); the diagnostics may only run on the two HPCs (a chain-level `ecus`).

Run.

```
mapping examples/uc2_redundant_chassis/application.json \
examples/uc2_redundant_chassis/configuration.json --cores
```

```
CADU: task-chains
```

chain	tasks	load %	path: segments on cores (sub-chains)	comm. ECU/net	reaction ms	deadline ms	on time	why late
steering_lane_A	4	33.00	c8@ZC_FL(0-1) > c16@ZC_FL(2-3)	1/0	4.4	8.0	yes	
steering_lane_B	4	33.00	c9@ZC_FR(0-1) > c17@ZC_FR(2-3)	1/0	4.3	8.0	yes	
braking_lane_A	3	24.00	c8@ZC_FL(0) > c16@ZC_FL(1) > c17@ZC_FR(2)	1/1	6.6	8.0	yes	
braking_lane_B	3	24.00	c11@ZC_RL(0) > c10@ZC_RR(1-2)	0/1	3.9	8.0	yes	
vehicle_dynamics	4	17.80	c11@ZC_RL(0-1) > c14@ZC_RR(2)	0/1	13.6	30.0	yes	
airbag	2	14.00	c17@ZC_FR(0-1)	0/0	1.4	3.0	yes	
suspension_control	3	8.00	c11@ZC_RL(0) > c10@ZC_RR(1) > c14@ZC_RR(2)	1/1	38.8	40.0	yes	
headlights	3	1.00	c8@ZC_FL(0) > c17@ZC_FR(1-2)	0/1	43.8	100.0	yes	
climate_control	3	0.80	c11@ZC_RL(0-1) > c10@ZC_RR(2)	0/1	413.7	1000.0	yes	

diagnostics	2	0.80	c0@HPC_A(0)	0/0	1008.8	-	yes

CADU keeps each lane in its own zones: the steering lanes never leave `ZC_FL` and `ZC_FR` (one hop between two cores of the zone controller, through its shared memory), so the replicas are on different ECUs by construction, and only the braking lanes cross the ring: two replicated flows. Every chain is on time with 8 cores.

The price of fewer messages.

metric	BFDU	WFDU	FFDU	CADU
cores used M	6	17	6	8
critical scaling factor	1.540	1.548	1.548	1.492
task-chains meeting deadline (%)	100.0	100.0	100.0	100.0
inter-ECU communications	12	14	12	6
replicated with FRER	5	8	5	2
network delay bound, max (us)	1725.3	4733.7	1725.3	291.7

BFDU and FFDU need only 6 cores, but they send the control tasks of both lanes to a central computer: twice as many messages, more than twice as many replicated flows, and network bounds six times larger. With `--separate-cores` the replicas may share an ECU (not a core): try it to see which constraint costs what. A tree topology is refused with `--frer`: a tree has no second route.

8.3 Use case 3: camera streams and control loops on one backbone

The question. Cameras (up to 120 KB per frame), radars and a lidar send their data from the zone controllers to the central computer, while 1 ms control loops cross the same switches between zones. Which traffic shaping keeps the control loops on time, and what happens with a slower link?

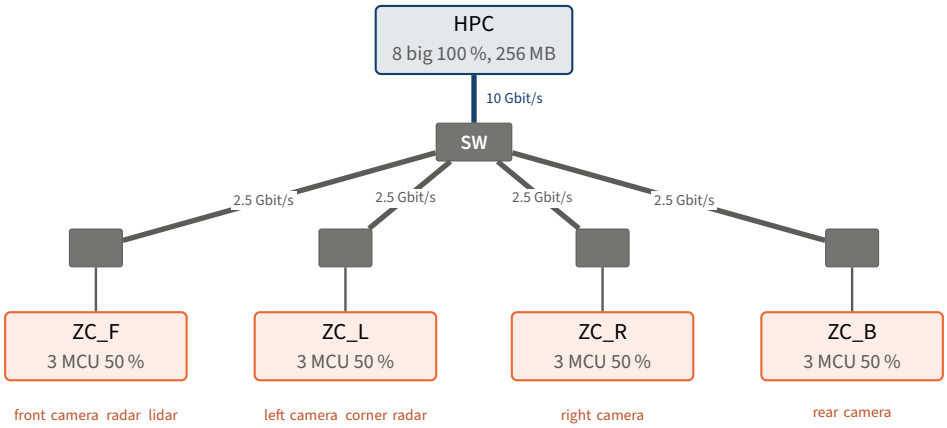


Figure 8. Use case 3: a tree rooted at the switch of the HPC; the zone links run at 1 Gbit/s, the switches are linked at 2.5 Gbit/s and the HPC at 10 Gbit/s.

The files. `examples/uc3_tsn_streams/`. The network model is a `tree` with `tree_root` `HPC`, `backbone_mbps` 2500 and `ecu_link_mbps` 10000 for the HPC; three traffic classes (producers up to 5 ms, up to 40 ms, the rest), TAS with a cycle of 500 µs and windows of 20, 40 and 20 %, frame preemption, and a design limit of 80 % per port. The application has 12 chains: perception streams (40 to 100 ms, messages of 3 to 120 KB) and control loops (1 to 20 ms, messages of 8 to 32 bytes).

Run, then change the network.

```
mapping examples/uc3_tsn_streams/application.json \
examples/uc3_tsn_streams/configuration.json [option]
```

```
options                  on time (%)    delay (us)      port (%)
(the file)                     100.0       14220.3          2.07
--ats                          100.0        2875.2          2.07
--classes 1                     75.0        4699.0          2.07
--link 100                      58.3       39008.3         20.70
--link 100 --ats                91.7       20256.9         20.70
```

With the file as it is, every chain is on time; the largest delay bound (14 ms) is the one of a camera frame of 120 KB, sent through the TAS window of its class. ATS (`--ats`) removes the waiting for the gates: the largest bound falls to 2.9 ms. With one class only (`--classes 1`) the messages of the 1 ms loops wait behind the camera frames: the steering and braking loops miss their deadline, and so does the front camera chain. With 100 Mbit/s links (`--link 100`) the ports carry 21 % of LET traffic and five chains are late; ATS saves four of them.

8.4 Use case 4: software updates on frozen hardware

The question. The hardware of a car is fixed at the start of production (SOP); its software keeps growing with updates for years. The platform of use case 1 is sized at the SOP with a margin; then the HPC variant is frozen. Do two later updates still fit, and how should they be mapped?

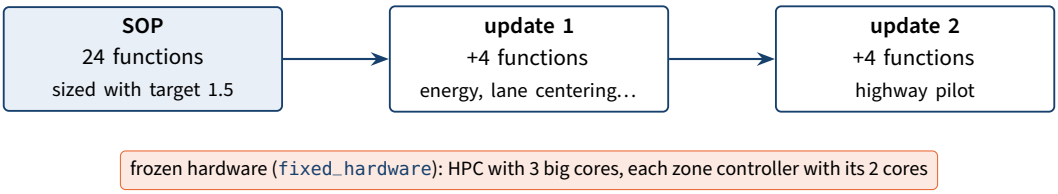


Figure 9. Use case 4: one platform, three versions of the software.

The files. `examples/uc4_software_updates/`: `application_sop.json` (the functions of use case 1), `application_update1.json` and `application_update2.json` (four more functions each, among them a highway pilot with 32 MB of perception); `configuration_sop.json` (use case 1 with `"target_scaling": 1.5` and 512 MB on the HPC) and `configuration_frozen.json` (the hardware chosen: 3 big cores on the HPC, at most the initial cores everywhere, `"fixed_hardware": true`).

Size at the SOP.

```
mapping examples/uc4_software_updates/application_sop.json \
examples/uc4_software_updates/configuration_sop.json
```

```
cores used per ECU, their load (in cores, LET included) and critical scaling factor
```

ECU	FFDU cores	FFDU load	FFDU scaling
HPC	2	1.07	1.779
ZC_FL	1	0.63	1.681
ZC_FR	1	0.61	1.759
ZC_RL	1	0.43	2.768
ZC_RR	1	0.15	7.059
ZC_C	1	0.10	12.438

The SOP needs 2 HPC cores for a scaling factor of 1.68; the carmaker buys a third one for the updates.

Map the updates on the frozen platform. Each version is mapped with the four rules on `configuration_frozen.json` (`--algorithm all`); the table gives the task-chains on time:

on time (%)	BFDU	WFDU	FFDU	CADU
sop	100.0	95.8	100.0	95.8
update1	92.9	100.0	92.9	85.7
update2	87.5	96.9	87.5	-

Both updates fit the frozen hardware with every rule but CADU, which cannot place a sensor task bound to `ZC_FL` in update 2: no core of that zone controller has room left, and the hardware is fixed. But the rules that minimize the cores (BFDU, FFDU) now leave chains late: packing the new functions moves the steering control off its zone controller. **On frozen hardware the cores are already paid for**: WFDU, which spreads the load over every core, keeps all chains of update 1 on time and 31 of 32 of update 2. The allocation objective changes after the SOP.

8.5 Use case 5: a hybrid platform with a big.LITTLE central computer

The question. A hybrid platform keeps an ADAS domain controller next to a central computer whose cores are of two types: 4 big and 4 little (at 40 % of the speed). The cores of each ECU share its memory. Where does each function go, and how much does memory contention cost?

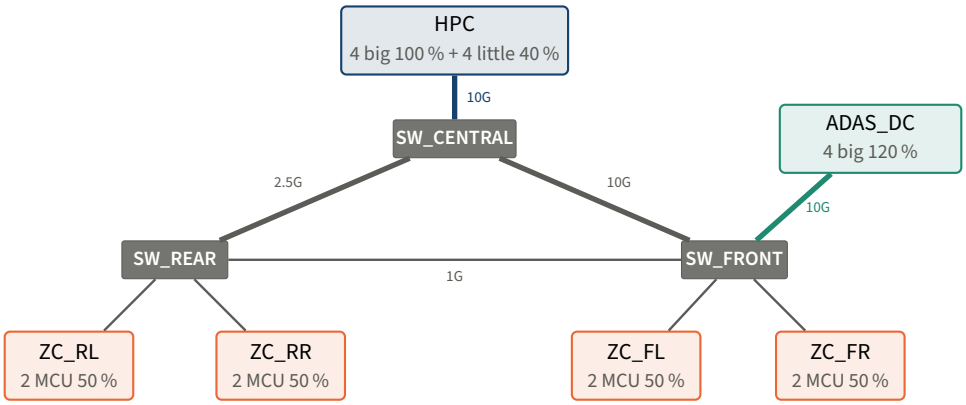


Figure 10. Use case 5: a custom backbone of three switches; the links of the HPC and of the ADAS controller run at 10 Gbit/s.

The files. `examples/uc5_hybrid_platform/`. The configuration has every section: the platform (a `central`, a `domain` and four `zone` ECUs, a 120 % ADAS controller), the interference model (40 ns per memory request), a `custom` topology (Section 5.4), ATS with two classes, and CADU. In the application the neural detector gains 30 % on big cores (`"wcet_factor_percent": {"big": 70}`) and may only run on the ADAS controller; the control laws lose less than the speed ratio on little cores (`{"little": 80}`); the battery and comfort strategies may only use little cores; every task gives its memory requests per job (`memory_accesses`).

Run.

```
mapping examples/uc5_hybrid_platform/application.json \
examples/uc5_hybrid_platform/configuration.json --cores
```

```
CADU: cores in use
```

core	ECU	type speed %	load %	LET %	scaling factor	sched	tasks	sub-chains chain.index
c0	HPC	big 100	35.4	0.1	2.830	yes	4	2.1 2.0 9.0
c4	HPC	little 40	16.0	0.6	6.480	yes	5	6.1 8.1 10.0 7.1
c8	ADAS_DC	big 120	41.3	0.4	2.318	yes	3	0.1 1.1 0.2
c13	ZC_FR	MCU 50	59.5	10.0	1.815	yes	5	3.2 1.0 8.0 3.1
c14	ZC_RL	MCU 50	50.5	10.0	2.223	yes	5	4.1 4.0 8.2 7.0 7.2
c15	ZC_RR	MCU 50	9.1	2.0	13.797	yes	4	6.0 5.2 2.2 6.2
c16	ZC_FL	MCU 50	48.7	10.0	2.324	yes	5	0.0 5.0 5.1 4.2 3.0

The perception runs on the ADAS controller (core c8), the planning on a big HPC core (c0), the strategies of battery, climate and body and the diagnostics on one little HPC core (c4); the control loops stay in the zone controllers. All chains are on time.

The cost of contention. `--memory-latency` overrides the interference model:

```
memory latency               cores  load (cores)       scaling   on time (%)
0 ns                             6          2.28         1.381         100.0
40 ns                            7          2.27         1.815         100.0
100 ns                           8          2.52         1.692         100.0
```

Without contention 6 cores suffice. At 40 ns per request, the WCETs grow with the number of cores competing for the memory of each ECU, and a seventh core is needed; at 100 ns an eighth one, and the application load grows by 10 %. The tool finds the number of competing cores of each ECU as a fixed point (more cores, more contention, longer WCETs).

9 The experiments

The experiments of the paper are programs, not hand-made runs: anyone can run them again and get the same files. This section explains, for each one, the question it answers, what varies, what stays fixed, what it measures and how to read its output.

9.1 How an experiment runs

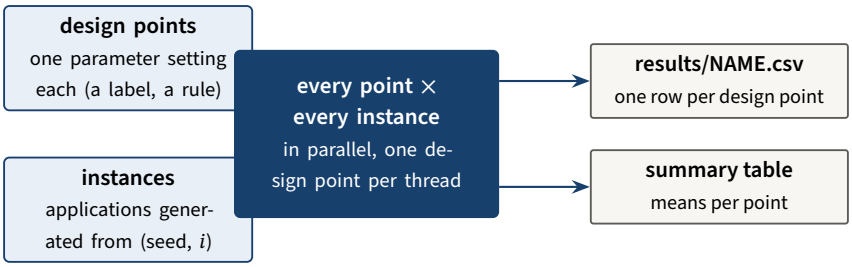


Figure 11. An experiment: every design point is run on the same generated applications, so that two points differ only by their parameters.

```
build/bin/experiment NAME [--instances N] [--jobs N] [--seed N] [--out FILE] [--quiet]
build/bin/experiment all   # every experiment, about three minutes on two cores
```

An experiment is a list of *design points*; each is run on several generated applications (the *instances*, 20 by default). Instance i always uses the application generated from (experiment seed, i): every point sees the same applications, and the results do not depend on the number of threads. Unless an experiment says otherwise, a point uses the defaults: 300 task-chains of the default generator, the built-in zonal platform (8 big cores on the PCU, 4 microcontroller cores at 50 % on each of the five zone controllers), $\varphi = 0.8$, $\sigma = 0.4$, LET tasks of 50 µs, a star at 1 Gbit/s with one CBS class, and every step of the method.

The program prints a progress line, then a table with the means of the metrics of the experiment over the instances of each point (over the feasible ones for the mapping metrics), and writes every design point to `results/NAME.csv` (Section 9.6). The columns of the tables:

column	meaning (mean over the instances)
feasible %	instances where every sub-chain found a core
cores M, lower bound	cores used, and the lower bound on them
PCU, ZCU cores	cores used on the central computers, on the other ECUs
FR %	fragmentation rate: cuts in % of the possible cuts
SR sched, SR %	schedulable cores after the priority assignment, and at the end
net hops, port max %	inter-ECU communications; LET load of the busiest port
on time, misses	task-chains meeting their deadline (%), late ones
reaction max	the largest maximum reaction time (ms)
scaling	critical scaling factor
CPU s	CPU time of a design point

9.2 Experiments of the method

`sweep` the sub-chain limit against the rule

Question. How does the limit σ trade cores against communications, and which rule packs best?

Varies. $\sigma \in \{0.02, 0.03, 0.05, 0.1, 0.2, 0.4, 0.8\}$ × the four rules: 28 points.

Measures. Cores and lower bound, cores per ECU kind, fragmentation rate, schedulable cores, chains on time, maximum reaction time, CPU time.

Reading. A small σ cuts chains into many small sub-chains: fewer cores (they pack like sand), but more communications and longer reaction times. Most chains use less than 8 % of a core, so the cuts happen below $\sigma \approx 0.1$. WFDU uses every core of the platform (28): it spreads on purpose.

`scaling` CPU time against the size

Question. Is the method fast enough for a whole vehicle?

Varies. 50 to 3200 task-chains (doubling) × the four rules; 2 instances per point.

Measures. Chains, tasks, feasibility, cores, schedulable cores, CPU time of the placement phases and of the whole design point.

Reading. The CPU time grows a little faster than linearly: about 2 s for 2000 chains on one core. The exact solver needs seconds to a minute for 300 chains (Section 9.8).

`architecture` speed of the zone controllers

Question. How fast must the zone controllers be?

Varies. Zone core speed 25, 50, 75, 100 % (PCU at 100 %) × the four rules.

Measures. Cores (total, PCU, zones), lower bound, schedulable cores, chains on time, LET load.

Reading. Slow zone cores take more cores for the tasks bound to the zones; the free sub-chains go to the PCU anyway.

`affinity` allocation with and without affinity

Question. Does trying the cores of the other sub-chains of the same chain first save communications?

Varies. $\sigma \in \{0.2, 0.4, 0.6\}$ × affinity on/off × the four rules.

Measures. Cores, communications inside an ECU and over the network, schedulable cores, chains on time, reaction time.

Reading. Compare each point with its twin without affinity: the same cores, fewer hops.

`ablation` each step of the method removed

Question. What does each step of Figure 3 bring?

Varies. Seven variants: the full method; without affinity; without consolidation; with the optional swap moves; without relocation moves; without the local search; without any of them (bin packing and the priority assignment only), on two settings, *default* and *stressed* (cores filled up to $\varphi = 1$, deadlines at 200 % of the pattern periods instead of 300 %), × FFDU and CADU: 28 points.

Measures. Cores, schedulable cores after the priorities and at the end, network hops, chains on time and late, CPU time.

Reading. Compare each variant with *full*. With the default setting, the consolidation saves cores (14.4 instead of 15.8 for FFDU) and the local search brings chains on time (92.6 instead of 88.4 %). The relocation moves only matter when cores are unschedulable, which happens in the stressed setting (without them, 0.7 % of the cores stay unschedulable). The swap moves change nothing, which is why they are off by default.

`practice` the method against practitioner rules

Question. Does the method beat what an engineer would do by hand?

Varies. The method with FFDU and with CADU; *whole chains* (cut only where the ECUs force it: $\sigma = \varphi$); *central only* (the free software on the central computer, the zones only for the tasks bound there); both together; *load balancing* (WFDU).

Measures. Cores (total, PCU, zones), lower bound, hops, busiest port, chains on time, reaction time.

Reading. Keeping chains whole costs little here (the chains are small), but see `limit`; putting all free software on the central computer costs cores, network traffic and deadlines; load balancing uses every core.

`limit` the sub-chain limit where it matters

Question. Why cut task-chains at all, and is the default $\sigma = 0.4$ right?

Varies. $\sigma \in \{0.4, 0.6, 0.8\}$ (0.8 is the cap: whole chains) on three settings: the default point, shared-memory contention at 50 ns, and 600 task-chains; × FFDU and CADU: 18 points.

Measures. Feasibility, cores and lower bound, hops, chains on time, CPU time.

Reading. At the default point a larger σ changes little (CADU saves about a core). With contention, the WCETs grow once the sub-chains are placed, and a large sub-chain no longer fits any core: only 45 % of the instances are feasible at $\sigma = 0.8$, 95 % at 0.6, all at 0.4. That is why the default is 0.4.

`deadlines` deadlines and the local search

Question. How tight can the deadlines be, and how much does the local search recover?

Varies. Deadlines at 200, 300, 400, 600 % of the sum of the pattern periods of each chain × local search on/off × the four rules.

Measures. Cores, chains on time, late chains, moves kept, reaction time, CPU time.

`fill` the utilization cap

Question. How high may the cores be filled?

Varies. $\varphi \in \{0.6, 0.7, 0.8, 0.9, 1.0\}$ × the four rules.

Measures. Cores, schedulable cores after the priorities and at the end, scaling factor, chains on time, CPU time of the placement.

Reading. Up to 0.9 the optimal priority assignment makes every core schedulable; at 1.0 about 1 % of the cores are not (the periods are not harmonic), and the relocation moves repair all of them.

`let` the cost of the LET tasks

Varies. WCET of the LET task: 10, 50, 100, 200 µs × the four rules.

Measures. Cores, LET load, communications on a core, schedulable cores, chains on time.

`communication` implicit communication or strict LET

Question. What does the determinism of strict LET cost, against implicit communication?

Varies. The two schemes on three settings: the default point, deadlines at 500 %, and a ring with three classes; × FFDU and CADU: 12 points.

Measures. Cores, hops, largest network bound, reaction times, chains on time.

Reading. The cores do not change. Strict LET removes the response time from the jitter of the flows, so the largest network bound falls by 15 to 19 %, but every crossing between cores waits for the end of the producer's period: 30 points fewer chains on time with FFDU, 16 with CADU, and only 4 and 1.5 with deadlines at 500 %.

`margins` WCET margin or sizing

Question. Two ways to keep a margin: raise every WCET at design time, or size for a scaling factor. Which one costs fewer cores for the same margin?

Varies. WCET margin 0, 10, 20, 40 %; then targets 1.10, 1.25, 1.50 for the sizing; × the four rules.

Measures. Cores, application load, schedulable cores, scaling factor, cap found, target reached, chains on time.

9.3 Experiments of the platform

`cores` types of cores

Question. What do little cores and lock-step cores change?

Varies. Four platforms: PCU with 8 big cores or 4 big + 8 little; zones with 4 microcontroller cores or 2 + 2 lock-step, the safety chains (10 %) on lock-step cores only; every WCET up to 30 % off its speed ratio on each type; × the four rules.

Measures. Cores (total, PCU, zones), application load, scaling factor, chains on time.

`constraints` spatial constraints

Varies. None; 10, 20, 30 % of replicated chains (on other ECUs); 20 % replicated on other cores only; 20 % of zone-only chains; 30 % of co-located tasks; 8 MB of memory per zone controller; × the four rules.

Measures. Cores (total, PCU, zones), hops, chains on time, reaction time.

Reading. Each constraint takes freedom from the allocation: its price is read in cores and hops against the point *none*.

`contention` shared-memory contention

Varies. Memory latency 0, 20, 50, 100 ns per request (1000 requests per ms of WCET), × the four rules.

Measures. Cores (total, PCU, zones), application load, schedulable cores, scaling factor, chains on time.

Reading. Contention grows with the cores used on an ECU, and more load needs more cores: a packed mapping pays the most.

`dse` four E/E architectures

Question. For the same vehicle functions, which architecture needs the fewest cores and the least network traffic, and up to what size of software does each one work?

Varies. The four templates of Figure 4 (distributed, domain, hybrid, zonal) × 100, 200, 300 task-chains × FFDU and CADU: 24 points. The generator draws the same functions for every template (the same seeds); their sensors and actuators are wired to the ECUs that are not central computers.

Measures. Feasibility, cores and lower bound, hops, busiest port, chains on time, reaction time, CPU time.

Reading. The distributed architecture runs out of cores as the software grows (each small ECU has at most two cores, the ADAS controller four): every instance is feasible at 100 chains, half of them with FFDU at 300. The domain, hybrid and zonal ones all absorb 300 chains; the hybrid and domain ones need about a core less than the zonal one, because part of the bound work runs on big cores.

`lifecycle` the SOP, then software updates

Question. How many updates does a core bought at the SOP buy?

Varies. The cap at the SOP: 0.5, 0.6, 0.7, 0.8 × the four rules. The platform is then fixed (the cores the SOP mapping uses), and ten updates of 15 new task-chains each are mapped on it at $\varphi = 0.8$, without moving what is already there.

Measures. Cores of the platform, scaling factor, updates absorbed, and after each update its chains, scaling factor and chains on time.

9.4 Experiments of the network

`network` link rate, preemption, message sizes

Varies. 100 or 1000 Mbit/s × frame preemption on/off × the default messages or sensor data (a third of large messages, up to 64 KB) × the four rules.

Measures. Cores, hops, largest network bound, busiest port, chains on time, reaction time.

`topology` star, tree and ring

Varies. Star; tree and ring with a backbone at 1 or 10 Gbit/s; tree and ring with ATS; × the four rules.

Measures. As for `network`.

Reading. On a ring the bursts of the flows grow at every hop and the dependencies between ports are cyclic; ATS reshapes every flow at every switch and removes that growth.

`classes` traffic classes on a whole vehicle

Varies. 2000 task-chains, 16 PCU cores; one CBS class at 1 Gbit/s; then with a 10 Gbit/s backbone: 1, 2 or 3 CBS classes, 1 or 3 TAS classes, 1 or 3 ATS classes; × the four rules; 2 instances.

Measures. Cores, largest bound, busiest port, chains on time and late, CPU time.

`frer` frame replication on the ring

Varies. CBS or ATS × 0, 10, 20, 30 % of replicated chains × FRER on/off × the four rules, on a ring.

Measures. Cores, hops, replicated flows, largest bound, busiest backbone link, chains on time.

9.5 Case studies and assumptions

`detection` a small detection chain

Varies. An illustrative ADAS chain of nine tasks (not taken from a benchmark; identical cores), $\varphi = 1$ and 0.8, σ from 0.2 to φ by 0.1, × the four rules; one instance.

Measures. Cores, schedulable cores, communications on a core, fragmentation rate, scaling factor.

`vehicle` the software of a whole vehicle

Varies. 2000 task-chains (about 8700 tasks), 16 PCU cores, at the reference speeds or with PCU cores four times faster (zones at 100 %); × the four rules; 2 instances.

Measures. Tasks, cores and lower bound, cores per ECU kind, schedulable cores, chains on time, busiest port, CPU time.

`robustness` the assumptions of the generator

Question. Do the conclusions depend on how the applications are generated?

Varies. One assumption at a time, below and above its default: chains local to one zone (0 or 50 % instead of 25), chains with a sensor and an actuator in a zone (30/25 or 90/80 % instead of 60/50), chain utilizations (halved or doubled), heavy chains (0 or 5 % instead of 2), message sizes (mostly small, or 40 % large); × FFDU and CADU: 22 points.

Measures. Cores and lower bound, hops, busiest port, chains on time, reaction time.

Reading. The values move, the ranking does not: at every point FFDU uses the fewest cores, and CADU the fewest network hops and the largest share of chains on time.

9.6 The CSV files

Every row starts with the design point and its parameters: `experiment, point, label, application, instance, seed, sigma, algorithm, phi, pcu_speed, zcu_speed, let_wcet, wcet_margin, affinity, repair, link_mbps, preemption, deadline_pct, error, topology, backbone_mbps, classes, shaping, frer, pcu_memory, zcu_memory, separation, replica_pct, colocate_pct, zone_only_pct, pcu_little, zcu_lockstep, type_variation, lockstep_pct, target_scaling, memory_latency, access_density`. The `label` tells the points apart when a parameter has no column of its own (the template of `dse`, the variant of `ablation`). The results follow.

columns	meaning
`chains, tasks, feasible`	size of the application; 1 if every sub-chain was placed
`fragments, cuts, forced_cuts, max_cuts, fr`	partitioning; fragmentation rate (%)
`cores_total, cores_arch, cores_used, cores_lb`	cores at the end, initial cores, cores used M, lower bound
`cores_pcu, cores_zcu, cores_big ... cores_lockstep`	M on the central computers and the other ECUs, and by core type
`sr_sched, sr_swap, sr_disp, sr_final`	schedulable cores after priorities, swaps, relocations, at the end (%)
`sr_chains, sr_on_time, deadline_misses`	task-chains schedulable and meeting their deadline (%); late ones
`repairs, emptied`	moves kept by the local search; cores emptied by consolidation
`u_app, u_let, u_unused`	application load, LET load, room below the cap (cores)
`hops_intra, hops_inter`	communications inside an ECU, between ECUs
`net_overloaded, net_avg, net_max`	flows without a bound; mean and largest flow bound (µs)
`port_max, backbone_max, ports_overloaded, frer_flows`	busiest port and backbone link (%); overloaded ports; replicated flows
`latency_avg/max, delay_avg/max, react_avg/max`	end-to-end response time, end-to-end delay, maximum reaction time (µs)
`scaling, sized_phi, target_met, sizing_runs, t_sizing`	critical scaling factor; result of the sizing
`cores_c`k`, u_app_c`k`, scaling_c`k`, memory_c`k	per ECU $k = 0..5$ (the first six ECUs of the platform)
`port_out_c`k`, port_in_c`k	load of the port of ECU k and of the switch port towards it (%)
`updates, update_chains, update_phi, absorbed, platform_cores`	lifecycle: updates tried, chains per update, cap, updates absorbed, cores of the platform
`life_scaling_`u`, life_on_time_`u`, life_load_`u	after the SOP ($u = 0$) and each absorbed update
`t_partition ... t_total`	CPU time of each phase and of the design point (s)

9.7 Against other methods: simulated annealing and a genetic algorithm

`baselines` the heuristic against metaheuristics

Question. Would a general-purpose search do better, given time?

How. `baselines results.csv trace.csv [--sizes L] [--seeds N] [--sa L] [--ga L] [--platform P] [--no-local-search]` maps the same applications (50, 100, 200, 300 chains, seeds 1 to 10, the default generator) on the platform `default`, `slow` (zone cores at 25 %), `constraints` (replicas, zone-only chains, 8 MB per zone controller) or `domain`, with the four rules and with two metaheuristics that solve the same allocation problem: the same sub-chains, the same constraints, the same schedulability test, the same first objective (the fewest cores).

- **Simulated annealing** (SA): from a first-fit mapping, moves of one sub-chain or exchanges of two, accepted with the Metropolis rule; budgets of $2 \cdot 10^4$, $2 \cdot 10^5$ and 10^6 moves.
- **Genetic algorithm** (GA): an individual is an order of the sub-chains, decoded by first fit; tournament selection, order crossover, swap mutation, elitism; 40 individuals; 5, 25 and 100 generations.

Both get the deadline local search of the heuristic on their best mapping (unless `--no-local-search`), so that the chains on time compare the searches. `include/mapping/search.h` gives every detail.

Measures. Per application and method: cores, lower bound, schedulable cores, chains on time, CPU time; the trace file has every improvement of the best mapping over time (the quality-against-time figure of the paper).

Reading. Compare with the optimum of the exact solver for the same applications (Section 9.8). On the default platform the rules take milliseconds, SA needs about 10^6 moves (seconds) to match them, and GA stays above them at 200 and 300 chains. On the domain platform and with slow zones, GA (tens of seconds) finds fewer cores than the rules; on the constrained platform no method comes close to the exact solver at 200 chains.

9.8 The exact solver

`exact` the optimality gap

Question. How far from the fewest possible cores is the heuristic?

How. The exact formulation (paper, section "Exact formulation") finds the fewest cores for the sub-chains of the partitioning, with the same constraints and schedulability test, as a constraint program (OR-Tools CP-SAT) with lazy schedulability cuts.

```
build/bin/export_instance 1 100 inst.json   # seed, task-chains, file
python3 exact/exact.py inst.json --time-limit 60 --json result.json
python3 exact/gap.py build/bin/export_instance --sizes 20,50,100,200,300 --seeds 10
python3 exact/gap.py build/bin/export_instance --sizes 20,50,100,200,300 --seeds 10 \
--configs domain,hybrid --out results/exact_architectures.csv
```

`export_instance` generates the application, partitions it once, runs the four rules (with and without consolidation) and writes the sub-chains, the candidate cores and the heuristic results. `exact.py` prints the status (`OPTIMAL`, `FEASIBLE` at the time limit, `INFEASIBLE`), the optimum and the time. `gap.py` solves 150 instances on the zonal platform (default, slow zones, constraints), checks every exact solution with the tool (`export_instance --verify`) and writes `results/exact.csv`; the second run adds 100 instances on the domain and hybrid templates (`export_instance --template T`). `--spare N` gives each ECU N candidate cores beyond the most any rule uses (2 by default); with 4, no instance gets a smaller optimum.

Reading. The gap of a rule is its cores minus the optimum, in % of the optimum.

9.9 Checking the bounds by simulation

The analysis gives upper bounds. Two simulators check that no simulated delay ever exceeds them.

`netsim` the network, frame by frame

How. `tests/netsim/netsim.py` simulates the frames of a flow set at every output port: store-and-forward switches, CBS with the idle slopes of the analysis, TAS gates, ATS regulators, FRER replication and elimination, a saturated best-effort class, release jitter. It compares every delay with its Network Calculus bound.

```
build/bin/net_export map 0.3 1 flows.csv --topology ring --classes 3 --frer
python3 tests/netsim/netsim.py flows.csv 20000 2   # horizon (µs), runs
sh tests/netsim/run_netsim.sh build/bin/net_export tests/netsim/netsim.py 20000 2
```

`net_export map <sigma> <seed>` exports the flows of a mapping, `heavy <load> <seed>` random flows up to a load of every port; `--config FILE` takes the platform and network of a configuration file. The script runs 23 flow sets (star, tree, ring, custom; CBS, TAS, ATS; FRER; the templates).

Reading. `max observed/bound` below 1 and `violations = 0` on every line.

`syssim` the whole system: cores, LET and network

How. `tests/syssim/syssim.py` simulates a mapping written by `--out` (`system.json`): every core under preemptive fixed priorities with its LET task, every LET copy (implicit or strict LET, as in the mapping), every message with a delay up to its network bound, the cores on unsynchronized phases. It follows each sensor sample through its chain to the first actuator job that uses it, and compares the end-to-end delay and the maximum reaction time of every chain with their bounds; half of the scenarios run every job at its WCET and every message at its bound, the others draw them lower.

```
build/bin/mapping --chains 200 --out run && \
python3 tests/syssim/syssim.py run/system.json 4
sh tests/syssim/experiment.sh build/bin/mapping results/syssim.csv 5 4
```

The experiment script maps five applications of 200 chains on nine settings (zonal star with FFDU and CADU, ring with FRER and three classes, memory contention, tree with TAS, strict LET on the star and on the ring, the domain and hybrid templates) and writes every chain's bounds and observed values to `results/syssim.csv`.

Reading. `violations = 0` on every line; the ratio observed/bound of each chain shows how tight the bounds are (1.000 for a chain on one core at its WCETs: the bound is reached).

10 Verification

`ctest` runs 18 tests; all must pass before any change is published.

test	what it checks
`unit_*` (12 programs)	each module alone: response times, the optimality of the priority assignment against brute force, CBS/TAS/ATS/FRER bounds computed by hand, the ring fixed point, routes on custom topologies, consolidation, contention, the lifecycle, the JSON reader, the files and their error messages, the templates, the metaheuristics
`golden`	the whole pipeline on seven applications, byte for byte against `tests/golden`
`roundtrip`	an application written with `--generate` and a configuration written with `--print-config` give exactly the results of the generator and the options
`invariants`	20 random configurations: response times, WCETs, loads, constraints and end-to-end bounds recomputed independently, and the incremental analysis after random moves
`network_bounds`	the network simulator on 23 flow sets: no delay above its bound
`system_bounds`	the full-system simulator on eight mappings (zonal, domain, distributed, custom; CBS, TAS, ATS; FRER; contention): no end-to-end delay or reaction time above its bound
`experiment_threads`	the same CSV file with one or several threads
`exact`	a small optimality-gap study, every exact solution checked by the tool (needs OR-Tools)

Changing the method. A change that alters any result makes the `golden` test fail. If the change is intended, regenerate the reference results with `build/bin/golden tests/golden` and say why in the commit message. The sanitizer build (Section 2) must also pass the tests.

11 Reproducing the paper

```
build/bin/experiment all
build/bin/baselines results/baselines.csv results/baselines_trace.csv
for p in slow constraints domain; do build/bin/baselines b_$p.csv t_$p.csv --platform $p;
done
(cat b_slow.csv; tail -n +2 b_constraints.csv; tail -n +2 b_domain.csv) >
results/baselines_platforms.csv
python3 exact/gap.py build/bin/export_instance --sizes 20,50,100,200,300 --seeds 10
python3 exact/gap.py build/bin/export_instance --sizes 20,50,100,200,300 --seeds 10 \
--configs domain,hybrid --out results/exact_architectures.csv
sh tests/netsim/run_netsim.sh build/bin/net_export tests/netsim/netsim.py 20000 2
sh tests/syssim/experiment.sh build/bin/mapping results/syssim.csv 5 4
cd paper && make figures && make
```

The first command writes the CSV files of the 24 experiments; the next three the metaheuristic comparison on the default platform and on the three harder ones (about twenty minutes each on two cores); the following two the exact results on the zonal platforms (about fifteen minutes) and on the domain and hybrid ones (about three minutes); the next two print the simulation checks and write `results/syssim.csv`; the last draws the figures, writes the generated tables (`paper/make_tables.py`) and builds `paper/ifupauline2026-system-level-mapping-automotive.pdf`. The results are deterministic:

the same commands give the same files on any machine and with any number of threads, CPU times aside.

The use cases and this manual.

```
cd manual && make listings && make
```

regenerates every output shown in Sections 7 and 8 from the files of `examples/`, then the manual.

12 The code

The library is in `src/` (C11, Linux kernel coding style, no global state: one design point per thread), its headers in `include/mapping/`, the programs in `tools/`.

file	role (paper section)
`model.c`	tasks, chains, cores; WCET on a core, with contention (System model)
`context.c`	the parameters of a design point, the platform and its ECUs
`json.c`, `io.c`	the JSON reader; the application and configuration files; the templates
`generator.c`	synthetic applications, replicas, the detection chain
`partition.c`	cutting into sub-chains (Step 1)
`let.c`	the LET task of a core and the core loads
`assign.c`	initial cores, allocation rules, memory reservation, contention update (Step 2)
`placement.c`	consolidation, relocation moves and the optional swap moves (Steps 3 and 5)
`sched.c`	response times, priority assignment, scaling factor (Timing analysis)
`repair.c`	deadline-driven local search (Step 6)
`communication.c`	segments, communications, end-to-end bounds, incremental analysis
`network.c`	topologies, routes, CBS/TAS/ATS service, FRER, fixed point
`stats.c`	metrics, verification, lower bound
`pipeline.c`	one design point, contention fixed point, sizing
`lifecycle.c`	the SOP and the software updates (Life of a platform)
`search.c`	simulated annealing and genetic algorithm (baselines)
`experiment.c`	parallel experiments and CSV output

Adding an experiment. In `tools/experiment.c`, write a function that fills a list of points with `add_point()` (one call per point and rule, setting the parameters to vary; `mp_params_template()` gives a template platform), and add a line to the `experiments[]` table with its name, description, default number of instances and the columns of its summary.

Adding a key to the files. Add the field to `struct mp_params` (`context.h`) with its default in `mp_params_default()`, read it in the right section of `io.c` (with its key in the list of known keys and its range), write it in `mp_write_configuration()`, and document it in this manual. The `roundtrip` and `unit_io` tests then check that what is written is read back.

13 Questions

Why does WFDU use so many cores? Worst fit puts every sub-chain on the emptiest core, so it uses every core of the platform. Its core count measures spreading, not need: use it on hardware that is already fixed (use case 4), not to size a platform.

A task-chain is late although all its tasks are schedulable. Look at its path (`--cores`): every communication between cores costs LET delays, and every inter-ECU one also a network transfer. A chain whose sensor and actuator are in different zones crosses the network at least once, whatever the mapping. Try CADU, a larger `--sigma`, or check that the deadline is not below the sum of the periods the LET semantics imposes.

The mapping is infeasible. Read the line below the summary table (Section 7.5): it names the sub-chain and the reason (memory, core types, largest number of cores, fixed hardware, or a sub-chain larger than a core).

`--frer` is refused. Frame replication needs two disjoint routes: use a ring or a custom topology with a second path.

Can I use my own sensor-to-ECU wiring with generated applications? Generate the file (`--generate`), then edit the `ecu` of the tasks; or write the application yourself.

Are the results exact? The bounds are worst cases: a schedulable core never misses a deadline, a network bound is never exceeded, a chain on time is always on time, under the models of Section 3 (fixed priorities, LET, the TSN shapers, round-robin memory). The simulators check this (Section 10). The number of cores is the result of a heuristic: within a few percent of the optimum on the applications of the paper (Section 9.8).

Where do the generated applications come from? From the statistics of the WATERS 2015 engine-management benchmark (periods, shares of the runnables, activation patterns); the appendix of the paper gives every rule. `--seed` selects one.